10. PCA'nın avantajları ve sınırlılıkları

Avantajları	Sınırlılıkları
Boyut sayısını azaltır.	Bilgi kaybı oluşturabilir.
Model eğitimini hızlandırabilir.	Doğrusal yapıların ötesini yakalayamaz.
Görselleştirmeyi kolaylaştırır.	Özellik ölçeklerinden etkilenir.
Tekrarlanan bilgiyi azaltabilir.	Yeni bileşenlerin yorumu zordur.
Gürültüyü azaltabilir.	Aykırı değerlere duyarlıdır.

Önemli: PCA en büyük varyansı seçer ancak yüksek varyans, sınıflandırma açısından en yararlı bilgi anlamına gelmez. Ders PDF'inin 18. sayfasındaki örnekte PC1 varyansın %90'ını taşımasına rağmen sınıfları ayıramaz; PC2 ise kalan %10 ile sınıfları ayırır.

IX. Bölüm SVD ve Düşük boyutlu Temsil

SVD ne işe yarar?



Şekil 1 — Veri matrisinden düşük boyutlu temsile geçiş.

* Örneğin 100 özellikli bir veri setimiz olsun. Bu özelliklerin bazıları birbiriyle ilişkiliyse 100 farklı sütun, 100 bağımsız bilgi anlamına gelmez.
* SVD bu tekrar eden veya birbirine benzeyen yapıları bulur.

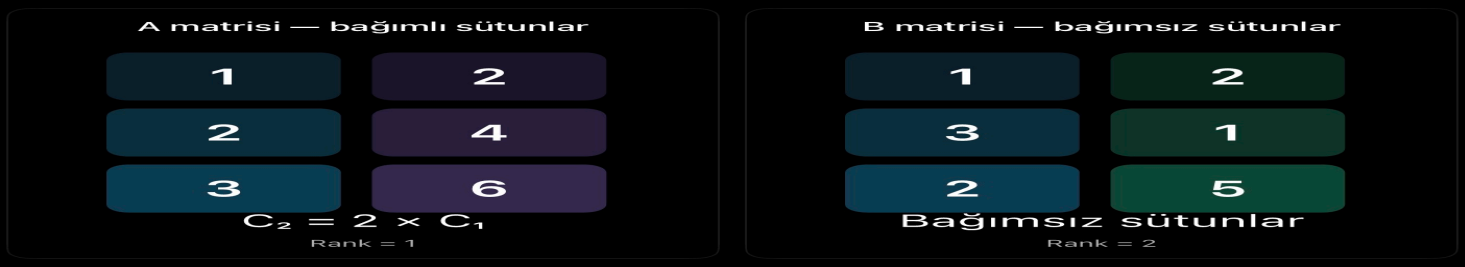
Redundancy - Gereksiz tekrar Eden bilgi

* Redundancy, bir veri kümesindeki aynı bilginin başka özellikler üzerinde yeniden üretilmesidir.

Örneğin;

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

- Bu matriste
1. sütun $[1,2,3]^T$
2. sütun $[2,4,6]^T$
İkinci sütun 1. sütunun 2 katıdır.
$C_1 = 2C_2$
Dolayısıyla bu iki sütun bağımsız veri taşımıyor.



İki matrisin boyutu da 3×2, ancak taşıdıkları bağımsız bilgi miktarı farklı.

Gerçek hayattan örnek
Bir veri setinde iki sütun olduğunu düşün:

Boy (metre)	Boy (cm)
1,70	170
1,80	180
1,65	165

Boyun santimetre cinsinden değeri, metre değerinin 100 katıdır. Bu yüzden iki değişken ayrı bilgiler değil, aynı bilginin farklı birimlerde gösterimidir.

Bu tür ilişkiler yüksek boyutlu verilerin daha düşük boyutta temsil edilebilmesinin temelini oluşturur.

Matris Rank - Matrisin Rütbesi

* Rank, bir matrisin lineer bağımsız veya sütunların sayısıdır.

$$rank(X) \leq \min(m, n)$$

- Burda:
  - m = satır sayısı
  - n = sütun sayısı

Örneğin 3x2 matrisin rank değeri maximum 2 olabilir.

Shape ve Rank farkı

	Shape	Rank
Anlam	Matrislerin fiziksel boyutu	Bağımsız bilgi yönü sayısı
Python	X.shape	np.linalg.matris_rank(X)
Örnek	(3,2)	1
Üst Sınır	Sütun ve satır	min(m,n)

Python Kodu

```
import numpy as np

A = np.array([
    [1, 2],
    [2, 4],
    [3, 6]
], dtype=float)

B = np.array([
    [1, 2],
    [3, 1],
    [2, 5]
], dtype=float)

print("A shape:", A.shape)
print("A rank:", np.linalg.matrix_rank(A))

print("B shape:", B.shape)
print("B rank:", np.linalg.matrix_rank(B))
```

Sistem Çıktısı

A shape: (3,2)
A rank = 1
B shape=(3,2)
B rank = 2

Low rank, fullrank ve Approximately Low Rank

Low rank	Rank, mümkün olan en büyük rank değerinden daha küçüktür.
full rank	Rank(X) = min(m,n)
Approximately low-rank	matris full rank olabilir, ama birkaç temel bileşen baskındır.

* Önemli: Gerçek veriler genellikle ölçüm hataları, gürültü ve küçük farklılıklar yüzünden tam rank çıkabilir.

Ama bu, boyut azaltmanın faydasız olduğu anlamına gelmez. Singular value değerinin çoğu küçükse matris yaklaşık olarak düşük rankla temsil edilir.

Örneğin; $\sigma = [100, 20, 12, 4, 1, 0.02, 0.5]$

- ilk birkaç değer diğerlerinden büyüktür. Baskın olduğu düşünülür.

5. Outer Product — Dış çarpım

SVD'nin temel yapı taşı olan rank-1 matrisi anlamak için dış çarpımı bilmeliyiz.

İki vektör tanımlayalım:

$$u = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \qquad v^T = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

Dış çarpımları:

$$uv^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

Python kodu:

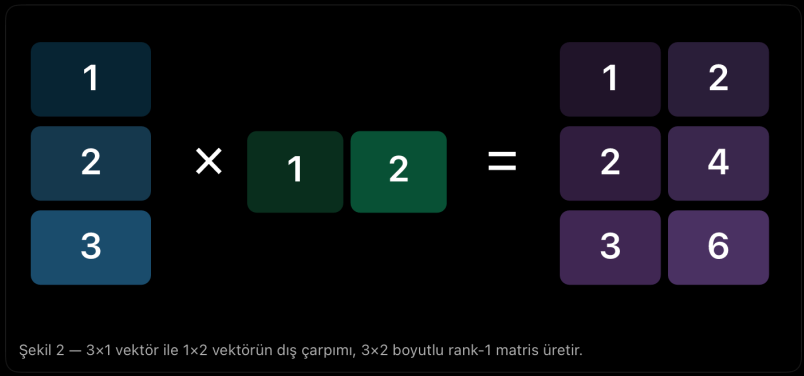
```python
import numpy as np

u = np.array([1, 2, 3])
v = np.array([1, 2])

A = np.outer(u, v)

print(A)
print("Rank:", np.linalg.matrix_rank(A))
```

Dot product ile outer product aynı şey değildir. Dot product iki vektörden skaler üretirken, outer product matris üretir.



Şekil 2 — 3×1 vektör ile 1×2 vektörün dış çarpımı, 3×2 boyutlu rank-1 matris üretir.

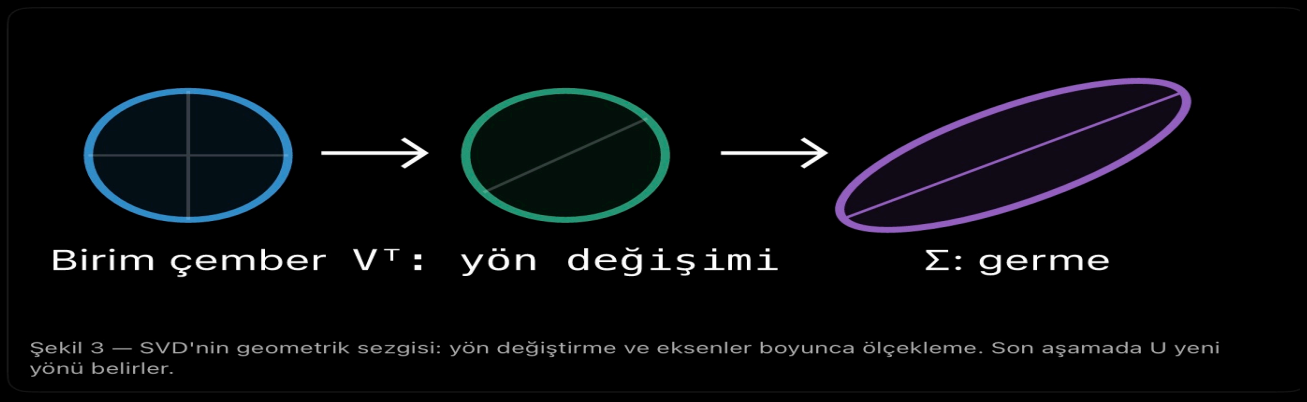
SVD - Singular Value Decomposition

* SVD bir matrisi üç parçaya ayırır:

$$X = U \cdot \Sigma \cdot V^T$$

Matris	Görevi
U	Solsingular vektörler; satır uzayındaki desenleri temsil eder.
Σ	Singular value'ların köşegeni tutar, bileşenlerin gücünü belirler
V^T	Sağ singular vektörler: sütun özelikleri uzayındaki desenleri takip eder.

* SVD'nin geometrik anlamı; bir vektör üzerinde önce yön değişimi, ardından eksenler boyunca ölçekleme, sonra yeniden yön değişimi gerçekleşir.



Şekil 3 — SVD'nin geometrik sezgisi: yön değiştirme ve eksenler boyunca ölçekleme. Son aşamada U yeni yönü belirler.

Singular value Nedir?

- Singular value, bir bileşenin matrise yaptığı katkı büyüklüğünü anlatır.

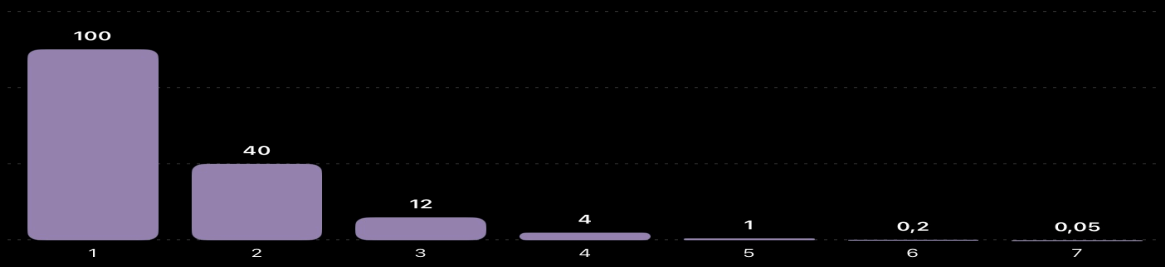
$$\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq \sigma_n$$

- Büyük singular value: Güçlü ve baskın yapı
- Küçük Singular value: Daha az enerji taşıyan yapı
- Sıfır Singular value: Bağımsız yeni yön oluşturmaz.

- Singular value negatif olmaz. Numpy `np.linalg.svd()` ile büyükten küçüğe sıralar

Singular Value Spectrum

Singular value spectrum
Ders notundaki örnek: ilk bileşenler diğerlerinden belirgin biçimde daha büyük.



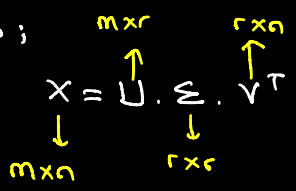
Grafik nasıl yorumlanır?

İlk üç bileşen güçlüdür, ardından değerler hızla küçülür. Bu noktaya yakın bir **dirsek (elbow)** bulunabilir. Dirsek, kaç bileşen tutulacağına karar verirken yardımcı olur; ancak her grafikte belirgin olmak zorunda değildir.

SVD matris boyutları

*X matrisi mxn boyutunda ve r = min(m,n) ise;

Reduced SVD;



Örnek:

$$X_{100\times 20} = U_{100\times 20}\Sigma_{20\times 20}V^T_{20\times 20}$$

Python'da:

```python
import numpy as np

X = np.random.default_rng(42).normal(size=(100, 20))

U, S, Vt = np.linalg.svd(
    X, full_matrices=False
)

print("X:", X.shape)
print("U:", U.shape)
print("S:", S.shape)
print("Vt:", Vt.shape)
```

Çıktı:

```
X: (100, 20)
U: (100, 20)
S: (20,)
Vt: (20, 20)
```

Burada dikkat: `S` bir matris değil, singular value'ları tutan tek boyutlu dizidir.

Σ matrisini oluşturmak için:

```python
Sigma = np.diag(S)
```

`full_matrices=False`, gereksiz büyük matrislerin üretilmesini önler. Özellikle satır sayısının çok yüksek olduğu veri setlerinde bellek açısından önemlidir.

Numpy ile adım adım SVD hesabı

$$X = \begin{bmatrix} 3 & 1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix}$$

* Singular value değeri:
  $\sigma_1 = 4.2426 \quad \sigma_2 = 2$
* Matrisin rankı: 2

```python
import numpy as np

X = np.array([
    [3, 1],
    [1, 3],
    [1, 1]
], dtype=float)

U, S, Vt = np.linalg.svd(
    X,
    full_matrices=False
)

print("U:")
print(np.round(U, 4))

print("Singular values:")
print(np.round(S, 4))

print("Vt:")
print(np.round(Vt, 4))
```

* Bütün bileşenleri geri oluşturma

$$\hat{X} = U \cdot \Sigma \cdot V^T$$

Python

```python
Sigma = np.diag(S)

X_hat = U @ Sigma @ Vt

print(np.round(X_hat, 4))
print(np.allclose(X, X_hat))
```

Çıktı:

Plain text

```
[[3. 1.]
 [1. 3.]
 [1. 1.]]
True
```

* np.allclose() sonucu true olduğuna göre yeniden oluşturulan matris sayısal hassasiyet içinde orjinal matrisle aynıdır.
* SVD uygulamak tek başına sıkıştırma değildir. Bütün bileşenleri kullanarak veri yeniden oluşturulur.

Rank-K Approximation – Düşük ranklı yaklaşım

* SVD ayrışımını başka şekilde yazılabilir:

$$X = \sum_{i=1}^{r} \sigma_i \cdot w_i \cdot v_i^T$$

Bu formül matris rank-1 bileşeni toplamı düşünmemizi sağlar.

* Yanlızca K bileşeni tutulursa:

$$\hat{X}_K = \sum_{i=1}^{K} \sigma_i \cdot w_i \cdot v_i^T$$

Önceki matriste $k = 1$ seçelim:

$$X_1 = \begin{bmatrix} 2 & 2 \\ 2 & 2 \\ 1 & 1 \end{bmatrix}$$

Orijinal ve yaklaşık matrisi karşılaştıralım:

Orijinal X

3	1
1	3
1	1

Rank = 2

Yaklaşım X_1

2	2
2	2
1	1

Rank = 1

Python:

```python
k = 1
U_k = U[:, :k]
S_k = S[:k]
Vt_k = Vt[:k, :]
X_k = U_k @ np.diag(S_k) @ Vt_k
print(np.round(X_k, 4))
```

İlk bileşen verinin baskın ortak yapısını koruyor, ikinci bileşenin temsil ettiği farkları ise kaybediyoruz.

İnteraktif: k değerini değiştir

Tutulan bileşen sayısı — k = 1

Rank-1 | Rank-2 (Tam)

Orijinal matris		Yaklaşım	
3	1	2	2
1	3	2	2
1	1	1	1

Frobenius hatası: 2.0000

Korunan enerji: 81.82%

İkinci bileşeni attığımız için 1. ve 2. sütunlardaki farklılıklar kayboldu.

İnteraktif: k değerini değiştir

Tutulan bileşen sayısı — k = 2

Rank-1 | Rank-2 (Tam)

Orijinal matris		Yaklaşım	
3	1	3	1
1	3	1	3
1	1	1	1

Frobenius hatası: 0.0000

Korunan enerji: 100%

Her iki bileşen de kullanıldığı için orijinal matris yeniden oluşturuldu.

Rekonstrüksiyon hatası - Frobenius Norm

* Sıkıştırmanın sonucunda ne kadar bilgi kaybettiğimizi ölçer.

Frobenius Norm: $\|X\|_F = \sqrt{\sum_i \sum_j x_{ij}^2}$

Rekonstrük hatası: $E = \|X - X_k\|_F$

Bağıl Hata: $E_{Bağıl} = \frac{\text{Rekonstrük hata}}{\text{Frobenius Norm}} = \frac{\|X - X_k\|_F}{\sqrt{\sum_i \sum_j x_{ij}^2}}$

Örneğin:

$$X - X_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 0 & 0 \end{bmatrix}$$

Karelerini toplayalım:

$$E_1 = \sqrt{1^2 + (-1)^2 + (-1)^2 + 1^2} = \sqrt{4} = \boxed{2}$$

Orijinal matrisin normu:

$$\|X\|_F = \sqrt{22} \approx 4.6904$$

Bağıl hata:

$$E_{\text{bağıl}} = \frac{2}{\sqrt{22}} \approx \boxed{0.4264}$$

Yani yaklaşık %42,64 bağıl rekonstrüksiyon hatası vardır. Bu değer doğrudan "%42,64 hücre yanlış" anlamına gelmez.

Python kodu:

```python
hata = np.linalg.norm(X - X_k, ord="fro")
orijinal_norm = np.linalg.norm(X, ord="fro")
bagil_hata = hata / orijinal_norm

print("Hata:", round(hata, 4))
print("Bağıl hata:", round(bagil_hata, 4))
```

Eckart - Young Teoremi

- SVD ile elde edilen k bileşen yaklaşımı Frobenius norm bakımından en iyi rank - k yaklaşımıdır.
- Başka bir ifadeyle, aynı rank sınırlaması altında daha düşük Frobenius hatasına sahip bir yaklaşım üretilemez.

$$E_k = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}$$

Enerji Birikim hesabı

* Matrisin toplam enerjisi:

$$\|X\|_F^2 = \sum_{i=1}^{r} \sigma_i^2$$

* Tek bir bileşenin enerjisi

$$\text{Enerji}(i) = \frac{\sigma_i^2}{\sum_j \sigma_j^2}$$

Örnek: $\sigma = [10, 5, 2, 1]$

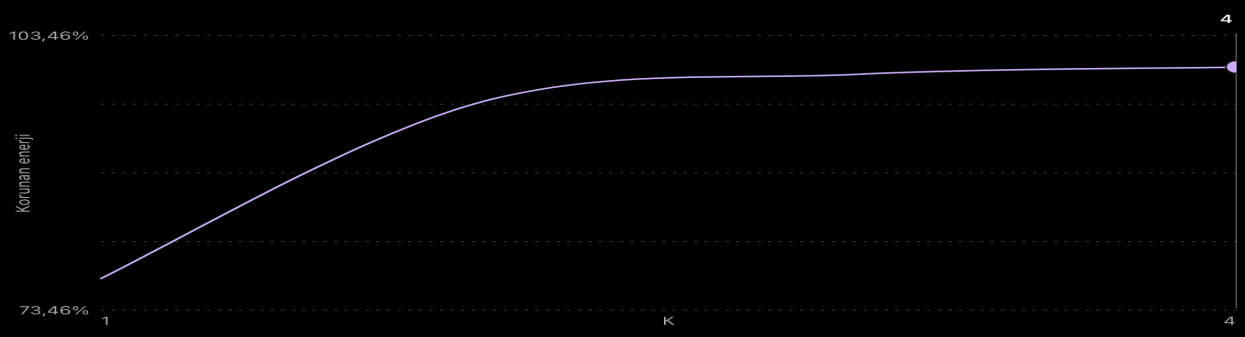
$\sigma^2 = [100, 25, 4, 1]$

Toplam enerji:

$$100 + 25 + 4 + 1 = 130$$

Bileşen	σ_i	σ_i^2	Enerji	Birikimli
1	10	100	%76,92	%76,92
2	5	25	%19,23	%96,15
3	2	4	%3,08	%99,23
4	1	1	%0,77	%100

Birikimli enerji eğrisi
İlk iki bileşen toplam enerjinin %96,15'ini taşıyor.



Soru: Enerjinin en az %90'ını korumak için minimum kaç bileşen gereklidir?

İlk bileşen %76,92 koruyor, yeterli değil. İlk iki bileşen %96,15 koruduğu için:

$$\boxed{k = 2}$$

Python ile otomatik k seçimi

Python ile otomatik k seçimi

Python

```python
import numpy as np

S = np.array([10, 5, 2, 1], dtype=float)

enerji = S**2
birikimli = np.cumsum(enerji) / enerji.sum()

for esik in [0.90, 0.95, 0.99]:
    k = np.searchsorted(birikimli, esik) + 1
    print(f"%{esik*100:.0f} enerji için k = {k}")
```

Çıktı:

Plain text

```
%90 enerji için k = 2
%95 enerji için k = 2
%99 enerji için k = 3
```

Burada iki önemli ilişki var:

$$\text{Korunan enerji} = 1 - (E_{\text{bağıl}})^2$$

$$E_{\text{bağıl}} = \sqrt{1 - C(k)}$$

Örneğin %96 enerji korunduğunda bağıl hata %4 değildir; yaklaşık %20'dir.

Bu, sınavda dikkat edilmesi gereken bir ayrıntıdır.

12. Görüntülerin matris olarak gösterimi

Ders notunun uygulama bölümünde bir görüntü, sayısal matris biçiminde temsil ediliyor.



Gri tonlamalı örnek görüntü.

Piksel matrisi

$$I = \begin{bmatrix} 0 & 85 & 120 \\ 45 & 180 & 230 \\ 70 & 210 & 255 \end{bmatrix}$$

0 siyah, 255 beyaz, ara değerler gri tonları temsil eder.

Ders PDF'inde kullanılan `skimage.data.camera()` görüntüsü:

- 512 × 512 pikseldir.
- Gri tonlamalıdır.
- 262.144 piksel değeri içerir.
- Ders notunda rank değeri 512 olarak verilmiştir.

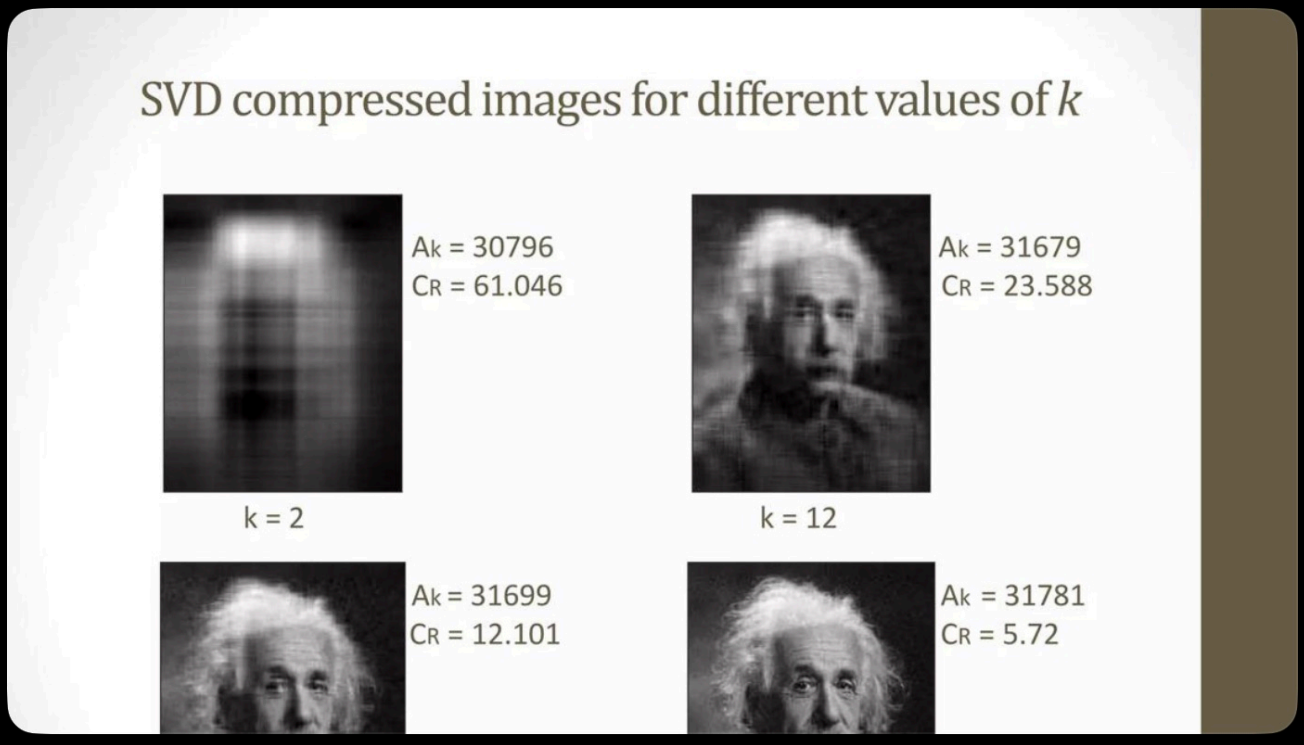
RGB görüntüler ise $H \times W \times 3$ yapısındadır. Bu haftanın örnekleri tek kanallı gri tonlamalı görüntüler üzerinde çalışıyor.

13. SVD ile görüntü sıkıştırma

Amaç, 512×512 görüntünün bütün bileşenlerini değil, en önemli ilk k bileşenini kullanarak görüntüyü yaklaşık biçimde yeniden oluşturmaktır.

$$I_k = U_k \Sigma_k V_k^T$$

k küçük olduğunda görüntünün kaba yapısı korunabilir ama ince ayrıntılar kaybolur. k büyüdükçe görüntü orijinaline yaklaşır.



Tam uygulama — Python

Bu kod PDF'deki kamera örneğini temel alır. Orijinal görüntüyü ve dört farklı düşük rank yaklaşımını yan yana çizer.

```python
import numpy as np
import matplotlib.pyplot as plt
from skimage import data

# 1. Görüntüyü yükle
img = data.camera().astype(float)

# 2. SVD ayrışımı
U, S, Vt = np.linalg.svd(
    img, full_matrices=False
)

# 3. Rank-k fonksiyonu
def reconstruct(k):
    return (U[:, :k] * S[:k]) @ Vt[:k, :]

# 4. Görselleştirme
k_values = [5, 10, 20, 50]

fig, axes = plt.subplots(1, 5, figsize=(17, 4))

axes[0].imshow(img, cmap="gray", vmin=0, vmax=255)
axes[0].set_title("Orijinal")

for ax, k in zip(axes[1:], k_values):
    img_k = reconstruct(k)
    ax.imshow(img_k, cmap="gray", vmin=0, vmax=255)
    ax.set_title(f"k = {k}")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Hata ve enerji tablosu oluşturma

Python

```python
m, n = img.shape
toplam = m * n
toplam_enerji = np.sum(S**2)

print("k | Sayı | Oran | Enerji % | Bağıl hata")

for k in [5, 10, 20, 50, 100]:
    img_k = reconstruct(k)

    saklanan = k * (m + n + 1)
    oran = toplam / saklanan

    enerji = np.sum(S[:k]**2) / toplam_enerji
    hata = np.linalg.norm(img - img_k, "fro")
    bagil = hata / np.linalg.norm(img, "fro")

    print(
        k,
        saklanan,
        round(oran, 2),
        round(enerji * 100, 2),
        round(bagil, 4)
    )
```

14. Compression Ratio — Sıkıştırma oranı

Dersin en önemli hesaplama formüllerinden biri:

Orijinal matris $m \times n$ boyutundadır.

$$N_{\text{orijinal}} = mn$$

Rank-k için U, S ve V^T bileşenlerinde saklanacak sayı adedi:

$$N_{\text{SVD}} = mk + k + kn$$

Dolayısıyla:

$$\boxed{CR = \frac{mn}{k(m+n+1)}}$$

Elle çözümlü örnek

512×512 görüntü ve $k = 20$ için:

Orijinal sayı adedi:

$$512 \times 512 = 262144$$

SVD temsili:

$$20(512 + 512 + 1) = 20500$$

Sıkıştırma oranı:

$$CR = \frac{262144}{20500} \approx 12.79$$

Yani idealize edilmiş bu sayı saklama modelinde yaklaşık **12,79:1** oranı elde edilir.

15. PCA ile SVD arasındaki ilişki

Önceki hafta öğrendiğin PCA ile SVD'nin önemli bir bağlantısı var.

PCA genellikle kovaryans matrisi üzerinden anlatılır:

$$C = \frac{1}{N-1} X_c^T X_c$$

Burada X_c, sütun ortalamaları çıkarılmış veri matrisidir.

Merkezlenmiş veriye SVD uygularsak:

$$X_c = U \Sigma V^T$$

PCA özdeğerleri ile SVD singular value'ları arasında:

$$\boxed{\lambda_i = \frac{\sigma_i^2}{N-1}}$$

ilişkisi bulunur.

PCA	SVD
Varyansı en büyük yönleri arar	Matrisi singular bileşenlere ayırır
Genellikle merkezlenmiş veride çalışır	Ham veya merkezlenmiş matrise uygulanabilir
Principal component yönleri	Sağ singular vektörlerle ilişkilidir
Explained variance	Merkezlenmiş veride singular value kareleriyle ilişkilidir

Python ile PCA-SVD bağlantısı

Python

```python
import numpy as np

X = np.array([
    [2, 4],
    [3, 6],
    [4, 8],
    [5, 10]
], dtype=float)

# Merkezleme
Xc = X - X.mean(axis=0)

# Kovaryans matrisi
C = np.cov(Xc, rowvar=False)

# PCA özdeğerleri
eigenvalues = np.linalg.eigvalsh(C)[::-1]

# SVD
U, S, Vt = np.linalg.svd(
    Xc, full_matrices=False
)

svd_eigenvalues = S**2 / (len(X) - 1)

print("PCA:", eigenvalues)
print("SVD:", svd_eigenvalues)
print(np.allclose(eigenvalues, svd_eigenvalues))
```